

```
In [1]: import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    f1_score, recall_score, precision_score, accuracy_score, roc_auc_score,
    average_precision_score, classification_report, confusion_matrix
)
from torch.utils.data import DataLoader, TensorDataset

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
In [22]: %pwd
```

```
Out[22]: 'C:\\Users\\anton\\Documents\\programming\\python\\ML\\QNN'
```

```
In [2]: # =====
# 1. Dataset
# =====
df = pd.read_csv("../creditcard.csv")

X = df.drop(columns=["Class"]).values
y = df["Class"].values
```

```
In [3]: scaler = StandardScaler()
X = scaler.fit_transform(X)

# índices fraude / não fraude
fraudes = np.where(y == 1)[0]
nao_fraudes = np.where(y == 0)[0]

# treino
train_fraudes = np.random.choice(fraudes, 390, replace=False)
train_nao_fraudes = np.random.choice(nao_fraudes, 1000, replace=False)

# teste
test_nao_fraudes = np.random.choice(
    np.setdiff1d(nao_fraudes, train_nao_fraudes), 1000, replace=False
)
test_fraudes = np.setdiff1d(fraudes, train_fraudes)

train_idx = np.concatenate([train_fraudes, train_nao_fraudes])
test_idx = np.concatenate([test_fraudes, test_nao_fraudes])

X_train, y_train = X[train_idx], y[train_idx]
X_test, y_test = X[test_idx], y[test_idx]

# tensores
X_train = torch.tensor(X_train, dtype=torch.float32).unsqueeze(1) # (batch,
seq=1, features)
y_train = torch.tensor(y_train, dtype=torch.float32).unsqueeze(1)
X_test = torch.tensor(X_test, dtype=torch.float32).unsqueeze(1)
y_test = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)

# DataLoaders
```

```

BATCH_SIZE = 64
train_loader = DataLoader(TensorDataset(X_train, y_train),
batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(TensorDataset(X_test, y_test), batch_size=BATCH_SIZE,
shuffle=False)

```

```

In [4]: # =====
# 2. LSTM à mão (Esposito)
# =====
class MyLSTM(nn.Module):
    def __init__(self, input_sz, hidden_sz, output_size=1):
        super().__init__()
        self.input_sz = input_sz
        self.hidden_size = hidden_sz

        # parâmetros
        self.W = nn.Parameter(torch.Tensor(input_sz, hidden_sz * 4))
        self.U = nn.Parameter(torch.Tensor(hidden_sz, hidden_sz * 4))
        self.bias = nn.Parameter(torch.Tensor(hidden_sz * 4))

        self.fc = nn.Linear(hidden_sz, output_size)
        self.sigmoid = nn.Sigmoid()
        self.init_weights()

    def init_weights(self):
        stdv = 1.0 / np.sqrt(self.hidden_size)
        for weight in self.parameters():
            weight.data.uniform_(-stdv, stdv)

    def forward(self, x):
        batch_sz, seq_len, _ = x.size()
        h_t = torch.zeros(batch_sz, self.hidden_size).to(DEVICE)
        c_t = torch.zeros(batch_sz, self.hidden_size).to(DEVICE)

        for t in range(seq_len):
            x_t = x[:, t, :] # (batch, input_sz)
            gates = x_t @ self.W + h_t @ self.U + self.bias
            i_t, f_t, g_t, o_t = torch.chunk(gates, 4, dim=1)

            i_t = torch.sigmoid(i_t)
            f_t = torch.sigmoid(f_t)
            g_t = torch.tanh(g_t)
            o_t = torch.sigmoid(o_t)

            c_t = f_t * c_t + i_t * g_t
            h_t = o_t * torch.tanh(c_t)

        out = self.fc(h_t)
        return self.sigmoid(out)

```

```

In [16]: # =====
# 3. Treinamento
# =====
model = MyLSTM(input_sz=X.shape[1], hidden_sz=32).to(DEVICE)
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=1e-4)

```

```

In [17]: v_loss = []

```

```
EPOCHS = 200
for epoch in range(EPOCHS):
    model.train()
    epoch_loss = 0
    for xb, yb in train_loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        optimizer.zero_grad()
        preds = model(xb)
        loss = criterion(preds, yb)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()
        v_loss.append(loss.item())

    print(f"Epoch {epoch+1}/{EPOCHS}, Loss: {epoch_loss/
len(train_loader):.4f}")
```

Epoch 1/200, Loss: 0.6634
Epoch 2/200, Loss: 0.6546
Epoch 3/200, Loss: 0.6463
Epoch 4/200, Loss: 0.6377
Epoch 5/200, Loss: 0.6293
Epoch 6/200, Loss: 0.6210
Epoch 7/200, Loss: 0.6124
Epoch 8/200, Loss: 0.6044
Epoch 9/200, Loss: 0.5959
Epoch 10/200, Loss: 0.5880
Epoch 11/200, Loss: 0.5806
Epoch 12/200, Loss: 0.5731
Epoch 13/200, Loss: 0.5654
Epoch 14/200, Loss: 0.5585
Epoch 15/200, Loss: 0.5516
Epoch 16/200, Loss: 0.5451
Epoch 17/200, Loss: 0.5391
Epoch 18/200, Loss: 0.5331
Epoch 19/200, Loss: 0.5274
Epoch 20/200, Loss: 0.5217
Epoch 21/200, Loss: 0.5158
Epoch 22/200, Loss: 0.5106
Epoch 23/200, Loss: 0.5053
Epoch 24/200, Loss: 0.5004
Epoch 25/200, Loss: 0.4948
Epoch 26/200, Loss: 0.4896
Epoch 27/200, Loss: 0.4855
Epoch 28/200, Loss: 0.4803
Epoch 29/200, Loss: 0.4752
Epoch 30/200, Loss: 0.4710
Epoch 31/200, Loss: 0.4656
Epoch 32/200, Loss: 0.4616
Epoch 33/200, Loss: 0.4567
Epoch 34/200, Loss: 0.4519
Epoch 35/200, Loss: 0.4478
Epoch 36/200, Loss: 0.4434
Epoch 37/200, Loss: 0.4384
Epoch 38/200, Loss: 0.4338
Epoch 39/200, Loss: 0.4295
Epoch 40/200, Loss: 0.4252
Epoch 41/200, Loss: 0.4210
Epoch 42/200, Loss: 0.4161
Epoch 43/200, Loss: 0.4122
Epoch 44/200, Loss: 0.4072
Epoch 45/200, Loss: 0.4030
Epoch 46/200, Loss: 0.3990
Epoch 47/200, Loss: 0.3942
Epoch 48/200, Loss: 0.3899
Epoch 49/200, Loss: 0.3860
Epoch 50/200, Loss: 0.3817
Epoch 51/200, Loss: 0.3774
Epoch 52/200, Loss: 0.3732
Epoch 53/200, Loss: 0.3685
Epoch 54/200, Loss: 0.3650
Epoch 55/200, Loss: 0.3606
Epoch 56/200, Loss: 0.3566
Epoch 57/200, Loss: 0.3527
Epoch 58/200, Loss: 0.3481
Epoch 59/200, Loss: 0.3441
Epoch 60/200, Loss: 0.3395

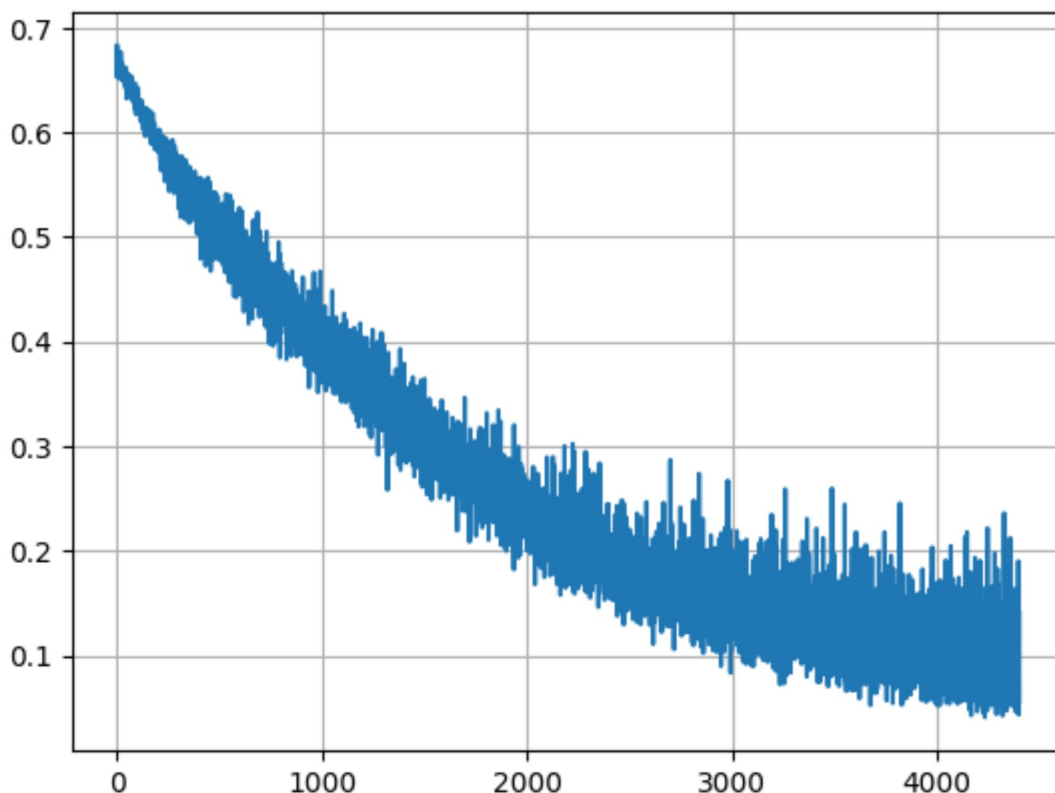
Epoch 61/200, Loss: 0.3362
Epoch 62/200, Loss: 0.3325
Epoch 63/200, Loss: 0.3284
Epoch 64/200, Loss: 0.3250
Epoch 65/200, Loss: 0.3217
Epoch 66/200, Loss: 0.3171
Epoch 67/200, Loss: 0.3136
Epoch 68/200, Loss: 0.3103
Epoch 69/200, Loss: 0.3069
Epoch 70/200, Loss: 0.3028
Epoch 71/200, Loss: 0.2990
Epoch 72/200, Loss: 0.2959
Epoch 73/200, Loss: 0.2918
Epoch 74/200, Loss: 0.2888
Epoch 75/200, Loss: 0.2850
Epoch 76/200, Loss: 0.2824
Epoch 77/200, Loss: 0.2791
Epoch 78/200, Loss: 0.2748
Epoch 79/200, Loss: 0.2718
Epoch 80/200, Loss: 0.2688
Epoch 81/200, Loss: 0.2649
Epoch 82/200, Loss: 0.2631
Epoch 83/200, Loss: 0.2591
Epoch 84/200, Loss: 0.2559
Epoch 85/200, Loss: 0.2528
Epoch 86/200, Loss: 0.2494
Epoch 87/200, Loss: 0.2475
Epoch 88/200, Loss: 0.2433
Epoch 89/200, Loss: 0.2414
Epoch 90/200, Loss: 0.2388
Epoch 91/200, Loss: 0.2358
Epoch 92/200, Loss: 0.2328
Epoch 93/200, Loss: 0.2298
Epoch 94/200, Loss: 0.2268
Epoch 95/200, Loss: 0.2244
Epoch 96/200, Loss: 0.2222
Epoch 97/200, Loss: 0.2193
Epoch 98/200, Loss: 0.2170
Epoch 99/200, Loss: 0.2144
Epoch 100/200, Loss: 0.2129
Epoch 101/200, Loss: 0.2099
Epoch 102/200, Loss: 0.2077
Epoch 103/200, Loss: 0.2055
Epoch 104/200, Loss: 0.2028
Epoch 105/200, Loss: 0.2009
Epoch 106/200, Loss: 0.1982
Epoch 107/200, Loss: 0.1973
Epoch 108/200, Loss: 0.1941
Epoch 109/200, Loss: 0.1927
Epoch 110/200, Loss: 0.1900
Epoch 111/200, Loss: 0.1881
Epoch 112/200, Loss: 0.1860
Epoch 113/200, Loss: 0.1835
Epoch 114/200, Loss: 0.1823
Epoch 115/200, Loss: 0.1797
Epoch 116/200, Loss: 0.1788
Epoch 117/200, Loss: 0.1762
Epoch 118/200, Loss: 0.1749
Epoch 119/200, Loss: 0.1736
Epoch 120/200, Loss: 0.1709

Epoch 121/200, Loss: 0.1692
Epoch 122/200, Loss: 0.1675
Epoch 123/200, Loss: 0.1671
Epoch 124/200, Loss: 0.1647
Epoch 125/200, Loss: 0.1627
Epoch 126/200, Loss: 0.1619
Epoch 127/200, Loss: 0.1601
Epoch 128/200, Loss: 0.1586
Epoch 129/200, Loss: 0.1571
Epoch 130/200, Loss: 0.1552
Epoch 131/200, Loss: 0.1540
Epoch 132/200, Loss: 0.1534
Epoch 133/200, Loss: 0.1522
Epoch 134/200, Loss: 0.1505
Epoch 135/200, Loss: 0.1497
Epoch 136/200, Loss: 0.1470
Epoch 137/200, Loss: 0.1464
Epoch 138/200, Loss: 0.1453
Epoch 139/200, Loss: 0.1440
Epoch 140/200, Loss: 0.1430
Epoch 141/200, Loss: 0.1424
Epoch 142/200, Loss: 0.1403
Epoch 143/200, Loss: 0.1396
Epoch 144/200, Loss: 0.1385
Epoch 145/200, Loss: 0.1371
Epoch 146/200, Loss: 0.1374
Epoch 147/200, Loss: 0.1344
Epoch 148/200, Loss: 0.1341
Epoch 149/200, Loss: 0.1334
Epoch 150/200, Loss: 0.1328
Epoch 151/200, Loss: 0.1321
Epoch 152/200, Loss: 0.1302
Epoch 153/200, Loss: 0.1291
Epoch 154/200, Loss: 0.1284
Epoch 155/200, Loss: 0.1283
Epoch 156/200, Loss: 0.1270
Epoch 157/200, Loss: 0.1258
Epoch 158/200, Loss: 0.1247
Epoch 159/200, Loss: 0.1236
Epoch 160/200, Loss: 0.1231
Epoch 161/200, Loss: 0.1227
Epoch 162/200, Loss: 0.1209
Epoch 163/200, Loss: 0.1202
Epoch 164/200, Loss: 0.1201
Epoch 165/200, Loss: 0.1201
Epoch 166/200, Loss: 0.1197
Epoch 167/200, Loss: 0.1170
Epoch 168/200, Loss: 0.1175
Epoch 169/200, Loss: 0.1165
Epoch 170/200, Loss: 0.1163
Epoch 171/200, Loss: 0.1159
Epoch 172/200, Loss: 0.1149
Epoch 173/200, Loss: 0.1132
Epoch 174/200, Loss: 0.1133
Epoch 175/200, Loss: 0.1123
Epoch 176/200, Loss: 0.1112
Epoch 177/200, Loss: 0.1108
Epoch 178/200, Loss: 0.1102
Epoch 179/200, Loss: 0.1096
Epoch 180/200, Loss: 0.1096

```
Epoch 181/200, Loss: 0.1080
Epoch 182/200, Loss: 0.1079
Epoch 183/200, Loss: 0.1081
Epoch 184/200, Loss: 0.1073
Epoch 185/200, Loss: 0.1069
Epoch 186/200, Loss: 0.1063
Epoch 187/200, Loss: 0.1054
Epoch 188/200, Loss: 0.1049
Epoch 189/200, Loss: 0.1048
Epoch 190/200, Loss: 0.1030
Epoch 191/200, Loss: 0.1037
Epoch 192/200, Loss: 0.1018
Epoch 193/200, Loss: 0.1018
Epoch 194/200, Loss: 0.1024
Epoch 195/200, Loss: 0.1009
Epoch 196/200, Loss: 0.1006
Epoch 197/200, Loss: 0.1003
Epoch 198/200, Loss: 0.0997
Epoch 199/200, Loss: 0.0986
Epoch 200/200, Loss: 0.0982
```

```
In [18]: import matplotlib.pyplot as plt
```

```
In [19]: plt.plot(v_loss)
plt.grid()
plt.show()
```



```
In [20]: # =====
# 4. Avaliação
# =====
model.eval()
y_true, y_pred, y_prob = [], [], []

with torch.no_grad():
    for xb, yb in test_loader:
```

```

xb, yb = xb.to(DEVICE), yb.to(DEVICE)
probs = model(xb)
preds = (probs >= 0.5).float()
y_true.extend(yb.cpu().numpy())
y_pred.extend(preds.cpu().numpy())
y_prob.extend(probs.cpu().numpy())

y_true = np.array(y_true).flatten()
y_pred = np.array(y_pred).flatten()
y_prob = np.array(y_prob).flatten()

```

```

In [21]: # métricas
f1 = f1_score(y_true, y_pred, average="macro")
recall = recall_score(y_true, y_pred, average="macro")
precision = precision_score(y_true, y_pred, average="macro")
accuracy = accuracy_score(y_true, y_pred)
auc = roc_auc_score(y_true, y_prob)
auc_pr = average_precision_score(y_true, y_prob)

cm = confusion_matrix(y_true, y_pred)
tn, fp, fn, tp = cm.ravel()
frr = fn / (fn + tp) if (fn + tp) > 0 else 0

print("\n=== Resultados ===")
print(f"F1-score (macro): {f1:.4f}")
print(f"Recall (macro): {recall:.4f}")
print(f"Precision (macro): {precision:.4f}")
print(f"Accuracy: {accuracy:.4f}")
print(f"AUC-ROC: {auc:.4f}")
print(f"AUC-PR: {auc_pr:.4f}")
print(f"FRR: {frr:.4f}")
print("\nConfusion Matrix:\n", cm)
print("\nClassification Report:\n", classification_report(y_true, y_pred,
digits=4))

```

=== Resultados ===

F1-score (macro): 0.9339
 Recall (macro): 0.9435
 Precision (macro): 0.9249
 Accuracy: 0.9773
 AUC-ROC: 0.9857
 AUC-PR: 0.9297
 FRR: 0.0980

Confusion Matrix:

```

[[985  15]
 [ 10  92]]

```

Classification Report:

	precision	recall	f1-score	support
0.0	0.9899	0.9850	0.9875	1000
1.0	0.8598	0.9020	0.8804	102
accuracy			0.9773	1102
macro avg	0.9249	0.9435	0.9339	1102
weighted avg	0.9779	0.9773	0.9776	1102

```

In [ ]: macro avg    0.9237    0.9386    0.9310    1102

```

weighted avg	0.9769	0.9764	0.9766	1102
--------------	--------	--------	--------	------